

Destruktor (*destructor*, *dtor*)

Gyakran előfordul, hogy egy objektum létrehozása során dinamikus erőforrásokat is (memória, állomány stb.) lefoglalunk, amelyeket az objektum megszűnésekor fel kell szabadítanunk. Ellenkező esetben ezek az erőforrások elvesznek a programunk számára.

A C++ nyelv biztosít egy speciális tagfüggvényt – a **destruktort** – amelyben gondoskodhatunk a lefoglalt erőforrások felszabadításáról. A destruktor nevét a hullám karakterrel (~) egybeépített osztály névként kell megadni. A destruktor a konstruktorhoz hasonlóan nem rendelkezik visszatérési típussal, nincsenek paraméterei, csak egy lehet belőle az osztályon belül. Ha magunk nem definiálunk destruktort, akkor a fordító a saját alapértelmezése szerinti változatot használja.

```
class A
{
public:
  A() { };  // konstruktor
  ~A() { }; // destruktor
};
```

Fontos: destruktor lehet virtuális, de nem lehet privát.

Másoló konstruktor (*copy constructor*, *cctor*)

A fordító alapértelmezetten sok kódot generál az osztályokba, struktúrákba. Konstruktoron és destruktoron kívül még másoló konstruktort is.

A másoló konstruktor egy olyan konstruktor, melynek egyetlen paramétere egy, az osztállyal azonos típusú objektum. Ez alapértelmezetten minden adattagot lemásol az adott adattag másoló konstruktora segítségével. Primitív típusoknál ez bitről bitre való másolást jelent. Azonban, ha az osztályunk dinamikusan létrehozott adattagokat is tartalmaz a fordító által generált másoló konstruktor már nem fogja tudni rendesen elvégezni a feladatát. Ugyanis nem fogja dinamikusan létrehozni az új változót csak egy mutatót, ami az eredeti objektum dinamikus változójára fog mutatni. Ennek pedig az a következménye, hogy az egyik objektumból képesek leszünk egy másik objektum privát adattagját módosítani, továbbá, ha az egyik osztálynak lefut a destruktora, ami felszabadítja a dinamikus változó által foglalt tárhelyet, amikor a következő osztály destruktora is lefut szintén megpróbál felszabadítani egy már felszabadított tárhelyet, ami nem definiált viselkedéshez vezet.

Ez azért van mert a fordító által létrehozott másoló konstruktor csak egy úgynevezett **sekély másolatot** (*shallow copy*) fog készíteni, azaz a dinamikus foglalású változóknak nem az értéke csak a hivatkozása lesz lemásolva. Az ilyen esetek elkerülése érdekében saját másoló konstruktor írása szükséges, ami **mély másolatot** (*deep copy*) készít.

```
class A
{
public:
  A() : _i(new int(1)) { }
  A(int i_) : _i(new int(i_)) { }
  ~A() { delete _i; }
private:
  int* _i;
};

int main()
{
  A a1(1);
  A a2(a1); // másoló konstruktor hívása
}
```

> kimenet: Error: double free or corruption

Itt egy sekély másolat fog készülni, ami azt jelenti, hogy a dinamikusan lefoglalt adattagok nem fognak újra létrejönni, hanem hivatkozni fognak az eredeti változóra, és amikor az `a2`-nek lefut a destruktora törölni fogja a dinamikusan lefoglalt értéket, ezért amikor az `a1` destruktora fog lefutni már egy nem létező változó értékét akarja majd törölni, ami nem definiált viselkedés. Írjunk hát saját másoló konstrukort.

```
class A
{
public:
  A() : _i(new int(1)) { }
  A(int i_) : _i(new int(i_)) { }
  ~A() { delete _i; }

  A(const A& rhs_) { _i = new int(*rhs_._i); }
private:
  int* _i;
};

int main()
{
  A a1(1);
  A a2(a1); // másoló konstruktor hívása
}
```

Az értékadó operátor (*assignmet operator*)

Hasonló a helyzet, mint a másoló konstruktor esetében, az alapértelmezetten létrehozott értékadó operátor is meghívja az egyes tagok értékadó operátorait, amik a primitív típusok esetén bitről bitre másolnak. Dinamikus típusok másolásához szükség van saját értékadó operátor írására. Az értékadó operátornak azonban van visszatérési értéke, mégpedig egy osztály típusú referencia, ami a többszörös értékadás miatt szükséges.

Tekintsük az előző programot, de a `main()`-t módosítsuk kicsit.

```
class A
{
public:
  A() : _i(new int(1)) { }
  A(int i_) : _i(new int(i_)) { }
  ~A() { delete _i; }

  A(const A& rhs_) { _i = new int(*rhs_._i); }
private:
  int* _i;
};

int main()
{
  A a1(1);
  A a2;
  a2 = a1; // értékadó operátor hívása
}
```

> kimenet: Error: double free or corruption

Hasonló a helyzet, mint a másoló konstruktor esetében, ezért hát írjunk saját értékadó operátort.

```
class A
{
public:
  A() : _i(new int(1)) { }
  A(int i_) : _i(new int(i_)) { }
  ~A() { delete _i; }

  A(const A& rhs_) { _i = new int(*rhs_._i); }

  A& operator=(const A& rhs_)
  {
    // régi memória felszabadítása
    delete _i;

    // ellenőrzés, hogy nem önértékadást csináltunk-e, ez az ellenőrzés
opcionális,
    // mivel viszonylag elég kicsi az esélye annak, hogy egy osztálynak
véletlenül
    // önmagát adjuk értékül.
```

```
    if (this != &rhs_)
    {
      _i = new int(*rhs_._i);
    }
    return *this;
  }
private:
  int* _i;
};

int main()
{
  A a1(1);
  A a2;
  a2 = a1; // értékadó operátor hívása
}
```

Fontos: a másoló konstruktor NEM értékadó operátor!

Gyakori hiba, hogy összekeverik a másoló konstruktor hívást az értékadó operátor hívásával.

```
int main()
{
  A a1;
  A a2(a1); //cctor
  A a3 = a2; // cctor
  A a4;
  a4 = a3; // értékadó operátor
}
```

Látható, hogy ha az értékadó operátort szeretnénk meghívi előbb létre kell hozni az objektum példányt, majd egy új sorban értékül adni neki egy már létező objektumpéldány értékeit.

Vexing parse

A **vexing parse** a szintaktikai félreérthetőség speciális formája a C++ programozási nyelnek. Ez akkor történik, amikor véletlenül függvénydeklarációt csinálunk függvény definíció helyett a default konstruktornál.

Tekintsük az alábbi programot:

```
class A
{
public:
  A() { }
};

int main()
{
```

```
  A a();
}
```

Jelen esetben nem konstruktorhívást végzünk, hanem egy függvény deklarációt csinálunk. Az `A a();` sor egy `a` nevű, bemeneti paraméternélküli, `A` visszatérési típusú függvény lesz.

RAII (Resource Acquisition is Initialization)

Ha olyan struktúrát, osztályt írtunk, amely gondoskodik arról, hogy minden dinamikusan lefoglalt területet felszabadít, mindent csak egyszer töröl, azt is jó sorrendben, akkor egy RAII struktúrát vagy osztályt írtunk. Ennek lényege, hogy az adott osztály a megfelelő erőforrásokat lefoglalja magának, majd a destruktor gondoskodik az erőforrások felszabadításáról. Minden erőforrást egy stack-en lévő objektumhoz kötünk, mivel azok garantáltan automatikusan fel fognak szabadulni, azaz destruktoruk le fog futni.

Bjarne Stroustrup híres mondása, hogy a C++ szemétgyűjtéssel rendelkező nyelv, mert nem generál szemetet.

`const` kulcsszó metódusok esetében

Korábban már láthattuk a `const` kulcsszó kettő lehetséges felhasználási módját függvények esetében.

- konstans visszatérési érték. Ebben az esetben a visszatérési értéken nem tudunk változtatni.

```
const std::string& f() { };
```

- konstans bemeneti paraméter. Ebben az esetben a bemeneti paraméter értéke nem módosulhat a függvényen belül.

```
void f(const std::string& i_) { }
```

Harmadik eset pedig amikor a metódust definiáljuk konstansnak

```
void f() const { }
```

Ez annyit jelent, hogy az `f()` függvényen belül az osztály adattagjainak értéke nem változhat meg. Továbbá fontos megjegyezni, hogy ha konstans objektumpéldányt hozunk létre, akkor annak csak konstans metódusait tudjuk elérni, ellenkező esetben módosítani tudnánk az adattagjait egy konstans objektumpéldánynak és így megsértenénk a konstans korrektséget. Továbbá konstans objektumpéldányok létrehozás esetében mindig kell, hogy legyen legalább egy, a felhasználó által definiált konstruktor.

```
#include <iostream>

class MyClass
```

```
{
public:
  MyClass() { _counter = 0; }
  void getCounter() const { std::cout << 1; }
  void getCounter() { std::cout << 2; }
private:
  int _counter;
};

int main()
{
  const MyClass mc;
  mc.getCounter();

  MyClass mc2;
  mc2.getCounter();
}
```

> kimenet: 12

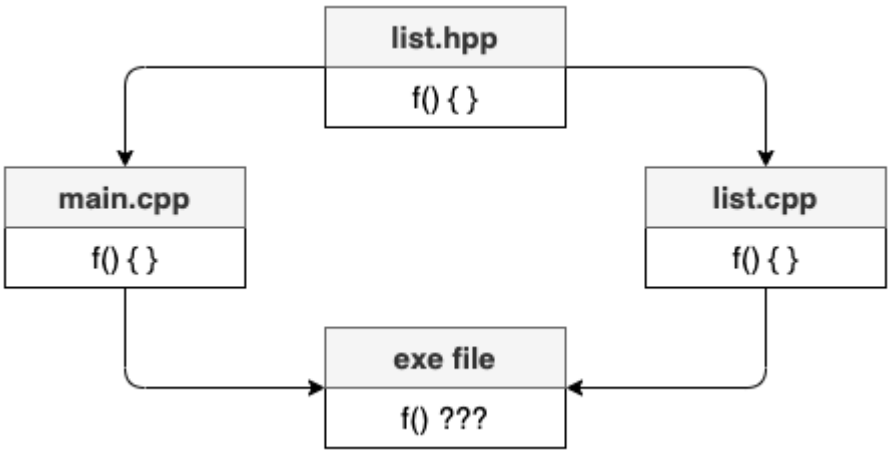
`inline` tagfüggvények

Az `inline` megoldás nagy előnye, hogy a teljes osztályt egyetlen fejállományban tárolhatjuk, és az osztály tagjait könnyen át tekinthetjük. Általában kisebb méretű függvények esetén alkalmazható hatékonyan. Ilyenkor nem függvényhívás történik, hanem magának a függvénynek a kódja beillesztésre kerül a hívások helyein. Nagyobb méretű kód: a függvény törzse több helyen szerepel, több optimalizálási lehetőség miatt azonban lehet mégis rövidebb. Megspórolja a függvényhívás idejét. Rekurzív függvények esetében nem használható, mert a fordító nem fogja tudni hányszor ágyazza egymásba az inline-olt kódot. Megadhatjuk explicit módon az `inline` kulcsszó segítségével vagy akár implicit módon is.

```
class MyClass
{
public:
  int getNumberI() const { return _i; } // implicit inline
  inline int getNumberJ() const { return _j; } // explicit inline
  // int inline és inline int között a fordító nem tesz különbséget
private:
  int _i;
  int _j;
};
```

Tekintsük azt a példát, amikor a `void f() {}` függvényt is beillesztjük a headerbe: ha több fordítási egységet fordítanánk egyszerre, melyben ez a header be van illesztve, linkelési hibát kapnánk, mert `f()` többször lesz definiálva. Ez azonban megkerülhető az inline kulcsszó használatával, segítségével ugyanis kiküszöbölhető a linkelési hiba: minden azonos nevű, visszatérési értékű, és paraméter listájú inline-ként definiált függvény definícióval együtt beilleszthető több különböző fordítási egységbe, és nem fog fordítási hibát okozni.

Ez úgy oldható meg, hogy a fordító a linkelés folyamán a definíciók közül egyet tetszőlegesen kivá- laszt. Az osztályon belül kifejtett függvények implicit inline-ok, így sose okozhatnak fordítási hibát.



Az fent látható ábra jól demonstrálja a problémát. `f()` egy ún. **strong reference**-el jön létre ha nem inline, így a linker hibát dob ha több fordítási egységben definiálva van. Ha azonban inline-ként adjuk meg, akkor **weak reference**-ként értelmezi, a meglevő definíciók közül tetszőlegesen kerül egy kiválasztásra. Ez nyilván azt is jelenti, hogy minden ilyen függvény definíciójának meg kell egyeznie, hisz kellemetlen meglepetés érhet minket, ha különböző definíciók közül olyat választ a fordító, melyre nem számítanánk (és ez egyben nem definiált viselkedés is).

Megjegyzés: a legtöbb fordítónál be lehet kapcsolni egy ún. **LTO** (*link time optimalization*) funkciót, amely a linkelésnél optimalizál, többek között ott végzi az inline-olást. Az inline függvények hajlamosak erősen megnövelni a bináris kódot, így az erőltetett használatuk nem javasolt. Az inline kulcsszó egy javaslat a fordító számára nem pedig parancs. Nem inline függvények lehetnek inline-ok, és inline-ként definiált függvények lehet mégsem lesznek azok.

`static` az osztályon belül

C++-ban az osztályszintű változókat és függvényeket a `static` prefixel kell ellátni. Ezek nem fognak létrejönni minden egyes objektum példánynál, mint az adattagok, hanem csak egyszer. Statikus adattagok kezdőértékkel való inicilizálása csak az osztályon kívül megengedett, kivétel az az eset, amikor konstans statikus adattagokról van szó. Ebben az esetben az osztályon belül tudunk nekik kezdőértéket adni. **Konstruktor és destruktor SOHA nem lehet statikus**.

Nem statikus metódusokból elérhetőek statikus és nem statikus adatagok is. Statikus metódosukból csak statikus adattagokra tudunk hivatkozni. Ennek oka, hogy ezek már fordítási időben létrejönnek, míg a nem statikus adattagok csak akkor, amikor példányosítjuk az osztályt.

```
class MyClass
{
public:
  int f() { return _i; } // helyes
  static int g() { return _k; } // hibás -> a return k; az implicit return
this->k;
                                // hívásnak felelne meg, ám az
osztályszintű metódus
```

```
                                    // esetében nincs értelmezve a this
mutató, hiszen
                                    // nincs objektum, amin értelmezhetnénk.
Statikus metódus
                                    // eléréséhez használjuk az osztály neévt
és a scope
                                    // operátort. MyClass::g()
private:
  int _k;
  static int _i; // osztályon kívül kell inicializálni
  const static int _j = 0; // lehet osztályon belül inicializálni, mert
konstans
};

int MyClass::_i = 0; // _i változó osztályon kívüli inicializálása
```

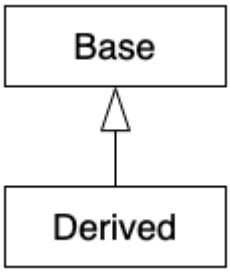
Öröklődésről bővebben

Mint azt már láthattuk a problémák objektum orientált feldolgozása során használatos az öröklődés. Lehetővé teszi, hogy már egy létező osztály vagy osztályok adatait és műveleteit új megközelítésben alkalmazzuk, illetve a feladat igényeinek megfelelően módosítsuk, bővítsük. Így nem egyetlen (nagy) osztállyal, hanem osztályok egymásra épülő rendszerével oldjuk meg a problémákat.

Az öröklődés az OO C++ nyelv egyik legfőbb sajátossága. Az öröklés során az új osztály örökli a meglévő osztály(ok) nyilvános (*public*) és védett (*protected*) adattagjait és tagfüggfényeit, amelyeket aztán annak sajátjaként használhatunk. Az új osztállyal bővíthetjük is a meglévő osztály(oka)t, új adattagokat és tagfüggvényeket definiálhatunk, illetve újra értelmezhetjük az örökölt, de működésükben elavult tagfüggvényket. Ez utóbbit nevezzük **override**-olásnak.

```
class Base { /* ... */ };

class Derived: public Base { /* ... */ }
```



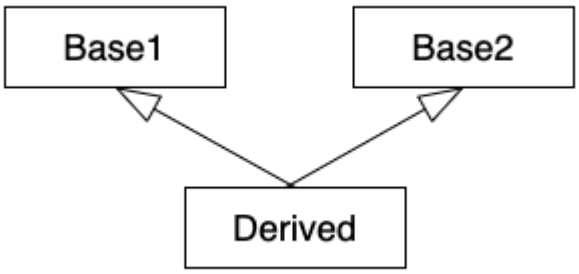
A C++ támogatja a többszörös öröklődést (*multiple inheritance*), mely során valamely új osztály több ősosztályból származik. Ez nem összekeverendő a több osztályon keresztüli örökléssel.

```
class Base1
{
public:
```

```
  int a;
};

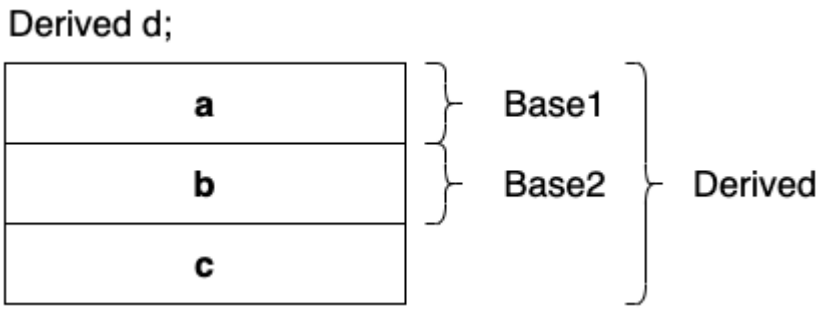
class Base2
{
public:
  int b;
};

class Derived: public Base1, public Base2
{
public:
  int c;
}
```



Az ábrán a `Derived` osztálynak két őse van: `Base1` és `Base2`. Egy osztálynak természetesen kettőnél több őse is lehet.

Nézzük meg, hogy a leszármazott `Derived` osztály objektumai hogyan néznek ki a memóriában.



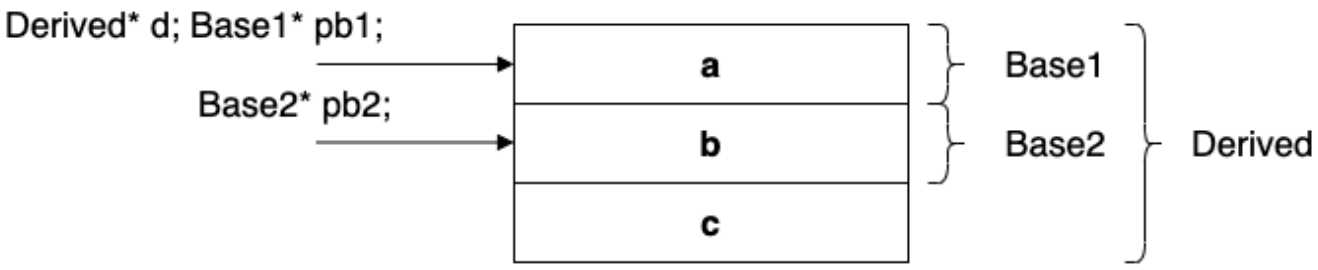
A `d` egy `Derived` osztálybeli objektum. Az objektumban először az örökölt tagváltozók foglalnak helyet, mégpedig a `Derived` osztály definíciójában megadott ősosztály listában balról jobbra haladva kerülnek be az objektumba: a `Base1` osztályból `a`, majd a `Base2` osztályból `b`. Ezt követik a leszármazott osztály tagváltozói, jelen esetben `c`.

Többszörös öröklés esetén is érvényes az a szabály, hogy leszármazott osztálybeli objektumra lehet ősosztály típusú mutatóval vagy referenciával hivatkozni. Természetesen itt is igaz, hogy ekkor úgy tekintünk az objektumra, mintha az adott őszosztálybeli lenne, ennek következtében csak az adott ősosztálybeli tagváltozók és tagfüggvények érhetőek el, illetve azok amelyeket az ősosztály örökölt a saját szüleitől. Tekintsük az alábbi kódot:

```
Derived d; //Derived őse Base1 és Base2
Derived* pd = &d;
Base1* pb1 = &d;
Base2* pb2 = &d;

pb1->a = 10;  // OK
pb1->b = 10;  // Fordítási hiba
pb2->a = 10   // Fordítási hiba
pb2->b = 10;  // OK
```

Bár az `a` és `b` változók jelen vannak a `Derived` osztályban, ettől függetlenül egy ősosztály típusú mutatóval csak azokat tudjuk elérni, melyek az adott ősosztályban is jelen vannak.



A legfontosabb tanulság a következő: **minden mutató a mutató típusának megfelelő rész elejére mutat az objektumban**. Első pillantásra talán kicsit furcsának tűnhet, hogy a `pb2` mutató nem az objektum elejére mutat, hiszen `Base2`-ből örökölt részek nem itt kezdődnek. Ennek fő következménye az, hogy **mutató konverzió esetén a mutató által tartalmazott cím megváltozhat.**

A származtatott osztály olyan osztály, amely adattagjait és függvényeit egy vagy több előzőleg definiált osztálytól örökli. A származtatott osztály szintén lehet alaposztálya további osztályoknak, lehetővé téve ezzel az osztályhierarchia kialakítását.

A származtatott osztály az alaposztály minden tagját örökli, azonban az alaposztályból csak a publikus és a védett tagokat éri el sajátként. A tagfüggvényeket általában publikus vagy védett hozzáféréssel adjuk meg, míg az adattagok esetén a védett vagy privát elérést alkalmazzuk. Öröklés során megadhatjuk a származtatás módját (**public**, **protected**,, **private**).

Az alaposztálybeli elérhetőségüktől függetlenül nem öröklődnek a konstruktorok, destruktorok, az értékadó operátor valamint a *friend* viszonyok.

Az öröklés módja	Alaposztálybeli elérés	Elérés a származtatott osztályban
public	public protected	public protected
protected	public protected	protected protected
private	public protected	private private

A publikus származtatás során az örökölt tagok megtartják az alaposztálybeli elérhetőségüket, míg privát származtatás során az örökölt tagok a származtatott osztály privát tagjaivá válnak. Védett öröklés esetén az örökölt tagok védettek lesznek az új osztályban.